

Rapport de projet informatique

KAMP : Application web de réseau social étudiant avec système de recommandation MBTI

Projet réalisé du 25/04/2026 au 24/05/2026

Membres du groupe

KESSILI EVA
AIT-ABAS YASMINE
TALBI MAISSA
BOUZIDI SHAYMA

Remerciements

Nous remercions notre enseignant pour son accompagnement durant ce projet informatique. Nous remercions également les membres du groupe pour leur implication dans la conception, le développement et les tests de l'application.

Table des matières

1	Introduction	4
2	Environnement de travail	4
3	Description du projet et objectifs	4
3.1	Présentation générale	4
3.2	Objectifs du projet	4
4	Bibliothèques, outils et technologies	5
4.1	React	5
4.2	Firebase	5
4.3	Firebase Authentication	5
4.4	Firestore Database	5
4.5	CSS	6
5	Travail réalisé	6
5.1	Authentification	6
5.2	Création du profil utilisateur	6
5.3	Fil d'actualité	6
5.4	Likes et commentaires	7
5.5	Notifications	7
5.6	Messagerie	8
5.7	Recommandation MBTI	8
5.8	Interface utilisateur	8
6	Difficultés rencontrées	9
7	Bilan	9
7.1	Conclusion	9
7.2	Perspectives	9
8	Bibliographie	10
9	Webographie	11
10	Annexes	12
A	Cahier des charges	12
B	Exemple d'exécution du projet	12
C	Manuel utilisateur	14

1 Introduction

Le projet KAMP consiste à développer une application web de réseau social destinée aux étudiants. L'objectif principal est de proposer une plateforme permettant aux utilisateurs de créer un profil, publier du contenu, communiquer avec d'autres étudiants et recevoir des recommandations de profils compatibles.

L'application s'inspire des réseaux sociaux modernes, mais elle ajoute une dimension originale grâce à un système de recommandation basé sur le profil MBTI des utilisateurs. Ce système permet de proposer des personnes potentiellement compatibles à partir du type de personnalité renseigné dans le profil.

Le projet a été développé avec React pour l'interface utilisateur et Firebase pour l'authentification et la gestion des données. Il comprend plusieurs fonctionnalités : inscription, connexion, création de profil, fil d'actualité, messages, notifications, commentaires, likes et recommandations MBTI.

2 Environnement de travail

Le projet a été réalisé dans un environnement de développement web moderne. Nous avons utilisé Visual Studio Code pour écrire et organiser le code source. Le projet a ensuite été sauvegardé et partagé grâce à GitHub.

L'application est une application web développée en JavaScript avec React. Elle fonctionne dans un navigateur web et utilise Firebase pour gérer les utilisateurs et les données.

Les principaux outils utilisés sont :

- Visual Studio Code pour le développement ;
- GitHub pour le stockage et le partage du code ;
- React pour construire l'interface utilisateur ;
- Firebase Authentication pour l'inscription et la connexion ;
- Firestore Database pour stocker les messages, posts et notifications ;
- CSS pour la mise en forme de l'application.

3 Description du projet et objectifs

3.1 Présentation générale

KAMP est une application web de réseau social étudiant. Elle permet à un utilisateur de se connecter, de créer son profil et d'interagir avec d'autres utilisateurs à travers un fil d'actualité, un système de messages et des notifications.

L'application est organisée en plusieurs composants React. Chaque composant correspond à une partie précise de l'interface : authentification, profil, feed, messagerie, notifications, barre latérale, recommandations MBTI, etc.

3.2 Objectifs du projet

Les objectifs principaux du projet sont les suivants :

- créer une application web interactive ;
- permettre l'inscription et la connexion des utilisateurs ;
- permettre la création d'un profil étudiant ;
- afficher un fil d'actualité avec des publications ;

- permettre aux utilisateurs de liker et commenter les publications ;
- créer une messagerie en temps réel ;
- afficher des notifications ;
- proposer un système de recommandation basé sur le MBTI ;
- obtenir une interface claire, moderne et simple à utiliser.

4 Bibliothèques, outils et technologies

Le projet utilise plusieurs bibliothèques et technologies.

4.1 React

React est utilisé pour construire l'interface utilisateur. L'application est divisée en composants indépendants, par exemple `Auth`, `Feed`, `Messages`, `Profile`, `PostCard` ou encore `AIMatchPage`.

React permet aussi de gérer les changements dynamiques dans l'application grâce aux hooks comme `useState`, `useEffect` et `useMemo`.

4.2 Firebase

Firebase est utilisé pour gérer la partie serveur du projet. Il permet de gérer l'authentification des utilisateurs et le stockage des données.

Dans le fichier `firebase.js`, l'application est connectée au projet Firebase avec le code suivant :

```

1 import { initializeApp } from "firebase/app";
2 import { getAuth } from "firebase/auth";
3 import { getFirestore } from "firebase/firestore";
4
5 const app = initializeApp(firebaseConfig);
6
7 export const auth = getAuth(app);
8 export const db = getFirestore(app);

```

4.3 Firebase Authentication

Firebase Authentication est utilisé pour créer des comptes utilisateurs et permettre la connexion. Dans le composant `Auth.jsx`, les fonctions utilisées sont :

```

1 createUserWithEmailAndPassword
2 signInWithEmailAndPassword

```

Ces fonctions permettent soit de créer un nouveau compte, soit de connecter un utilisateur existant.

4.4 Firestore Database

Firestore est utilisé pour stocker les données de l'application, notamment :

- les publications ;
- les messages ;

- les notifications ;
- les commentaires ;
- les likes.

La fonction `onSnapshot` permet d'écouter les modifications en temps réel. Ainsi, lorsqu'un message ou une publication est ajouté, l'affichage peut se mettre à jour automatiquement.

4.5 CSS

Le fichier `App.css` permet de gérer l'apparence de l'application. Il définit le design général : barre latérale, cartes de publications, messages, profil, boutons, couleurs et mise en page.

5 Travail réalisé

5.1 Authentification

Nous avons réalisé un système d'inscription et de connexion grâce à Firebase Authentication.

Le composant `Auth.jsx` contient un formulaire permettant à l'utilisateur de saisir son adresse email et son mot de passe. Selon le mode choisi, l'utilisateur peut soit se connecter, soit créer un nouveau compte.

```
1 await signInWithEmailAndPassword(auth, email, password);
2
3 await createUserWithEmailAndPassword(auth, email, password);
```

Cette fonctionnalité est essentielle car elle permet de sécuriser l'accès à l'application.

5.2 Création du profil utilisateur

Après la connexion, l'utilisateur doit créer son profil. Le composant `SetupProfile.jsx` permet de renseigner :

- un prénom ;
- un type MBTI ;
- une licence ou formation ;
- une photo de profil.

Les données du profil sont sauvegardées dans le `localStorage`. Cela permet de conserver les informations même après un rechargement de la page.

```
1 localStorage.setItem(
2   "kamp-profile",
3   JSON.stringify(profile)
4 );
```

5.3 Fil d'actualité

Le composant `Feed.jsx` permet d'afficher les publications des utilisateurs. Les publications sont récupérées depuis Firestore grâce à `onSnapshot`.

```

1 const unsubscribe = onSnapshot(
2   collection(db, "posts"),
3   (snapshot) => {
4     const loadedPosts = snapshot.docs.map((doc) => ({
5       id: doc.id,
6       ...doc.data()
7     }));
8
9     setPosts(loadedPosts.reverse());
10  }
11 );

```

L'utilisateur peut publier un message dans le feed. Chaque publication contient le nom de l'utilisateur, son MBTI, sa formation, son texte, son image éventuelle, ses likes et ses commentaires.

5.4 Likes et commentaires

Le composant `PostCard.jsx` gère l'affichage d'une publication. Il permet à l'utilisateur de liker un post et d'ajouter un commentaire.

Pour les likes, nous utilisons la fonction `increment` de Firebase :

```

1 await updateDoc(
2   doc(db, "posts", post.id),
3   {
4     likes: increment(1)
5   }
6 );

```

Pour les commentaires, nous utilisons `arrayUnion`, qui permet d'ajouter un commentaire à la liste des commentaires déjà présents.

```

1 await updateDoc(
2   doc(db, "posts", post.id),
3   {
4     comments: arrayUnion(newComment)
5   }
6 );

```

5.5 Notifications

Le composant `Notifications.jsx` affiche les notifications enregistrées dans Firestore. Une notification est créée lorsqu'un utilisateur aime ou commente une publication.

Exemple de création d'une notification :

```

1 await addDoc(collection(db, "notifications"), {
2   from: profile.username,
3   text: "a_aim_ ton_post",
4   createdAt: serverTimestamp()
5 });

```

Cette fonctionnalité permet de rendre l'application plus interactive.

5.6 Messagerie

Le composant `Messages.jsx` permet de créer une messagerie avec plusieurs salons :

- Général;
- Révisions;
- Gaming;
- Vocal nuit.

Les messages sont stockés dans Firestore dans une collection correspondant au salon choisi.

```
1 collection(db, "rooms", currentRoom, "messages")
```

La fonction `onSnapshot` permet d'afficher les messages en temps réel.

5.7 Recommandation MBTI

Le système de recommandation est une partie importante du projet. Il est présent dans les composants `AIMatch.jsx` et `AIMatchPage.jsx`.

L'utilisateur renseigne son type MBTI lors de la création de son profil. Ensuite, l'application utilise un tableau de correspondances pour proposer des profils compatibles.

Exemple :

```
1 INTJ: [  
2   {  
3     name: "Emma",  
4     mbti: "ENFP",  
5     compatibility: 92  
6   },  
7   {  
8     name: "Nathan",  
9     mbti: "ENTP",  
10    compatibility: 88  
11  }  
12 ]
```

Le programme récupère le MBTI de l'utilisateur avec :

```
1 profile?.mbti?.toUpperCase()
```

Puis il affiche les profils recommandés avec leur pourcentage de compatibilité.

Cette fonctionnalité montre que l'application ne se limite pas à un réseau social classique. Elle propose aussi une recommandation personnalisée selon le profil de l'utilisateur.

5.8 Interface utilisateur

L'interface est organisée autour d'une barre latérale qui permet de naviguer entre les différentes pages :

- Feed;
- Messages;
- Notifications;
- IA Match;

— Déconnexion.

Le composant `Sidebar.jsx` permet cette navigation.

6 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées pendant le projet.

La première difficulté a été la compréhension de Firebase. Il fallait comprendre comment connecter l'application à Firebase, comment utiliser l'authentification et comment stocker des données dans Firestore.

La deuxième difficulté a été la gestion des états avec React. L'application utilise plusieurs états pour stocker les messages, les posts, les profils, les commentaires ou encore la page actuellement affichée.

Une autre difficulté a été la messagerie en temps réel. Il fallait faire en sorte que les messages apparaissent automatiquement sans recharger la page.

Enfin, l'organisation du code a également demandé du travail. Comme l'application contient plusieurs composants, il fallait bien séparer les fichiers pour rendre le projet plus clair.

```
1 REPARTITION DU TRAVAIL :  
2 - Yasmine : 15%  
3 - Eva : 30%  
4 - Shayma : 30%  
5 - Maïssa : 25%
```

7 Bilan

7.1 Conclusion

Ce projet nous a permis de développer une application web complète avec React et Firebase. Nous avons appris à construire une interface avec plusieurs composants, à gérer l'authentification des utilisateurs et à manipuler une base de données en temps réel.

L'application KAMP possède plusieurs fonctionnalités importantes : connexion, profil, publications, likes, commentaires, notifications, messagerie et recommandations MBTI. Le projet répond donc aux objectifs fixés au départ.

7.2 Perspectives

Plusieurs améliorations pourraient être ajoutées à l'application :

- améliorer le système de recommandation avec un algorithme plus avancé ;
- stocker les profils directement dans Firestore au lieu du `localStorage` ;
- ajouter une vraie page de modification du profil ;
- améliorer la sécurité des règles Firebase ;
- rendre l'application entièrement responsive pour mobile ;
- ajouter un système de recherche d'utilisateurs ;
- ajouter des images stockées dans Firebase Storage.

8 Bibliographie

[REACT] React, Documentation officielle, Meta.

[FIREBASE] Firebase, Documentation officielle, Google.

9 Webographie

[REACTWEB] <https://react.dev/>

[FIREBASEWEB] <https://firebase.google.com/>

[GITHUB] <https://github.com/>

10 Annexes

Annexe A : Cahier des charges

Le cahier des charges du projet était de créer une application web fonctionnelle autour d'un réseau social étudiant.

Les fonctionnalités prévues étaient :

- inscription et connexion ;
- création d'un profil utilisateur ;
- publication de messages ;
- affichage d'un fil d'actualité ;
- likes et commentaires ;
- notifications ;
- messagerie ;
- recommandations de profils compatibles.

Les fonctionnalités réalisées sont :

- inscription et connexion avec Firebase ;
- création de profil avec nom, MBTI, formation et photo ;
- feed de publications ;
- likes ;
- commentaires ;
- notifications ;
- messagerie en temps réel ;
- recommandations MBTI ;
- interface avec barre latérale.

Annexe B : Exemple d'exécution du projet

Pour montrer le fonctionnement de l'application, il est possible d'ajouter ici des captures d'écran :

- page de connexion ;
- page de création de profil ;
- fil d'actualité ;
- publication d'un post ;
- ajout d'un commentaire ;
- réception d'une notification ;
- affichage des recommandations MBTI ;
- messagerie.

Exemple de scénario d'utilisation :

1. L'utilisateur ouvre l'application KAMP.
2. Il crée un compte avec une adresse email et un mot de passe.
3. Il complète son profil avec son prénom, son MBTI et sa formation.
4. Il accède au feed.
5. Il publie un message.
6. Un autre utilisateur peut liker ou commenter la publication.
7. Une notification est créée.

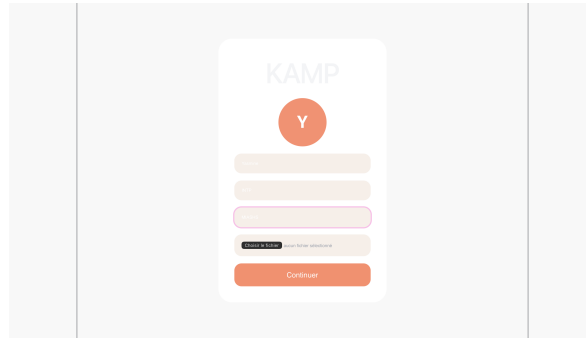


FIGURE 1 – Page de connexion

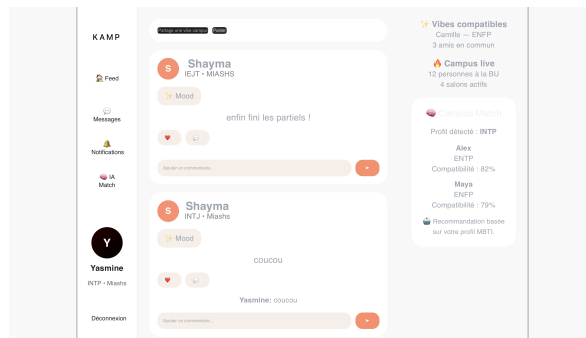


FIGURE 2 – Feed de l'application

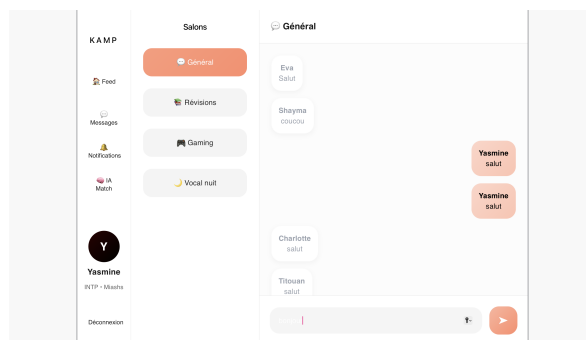


FIGURE 3 – Salons de discussions

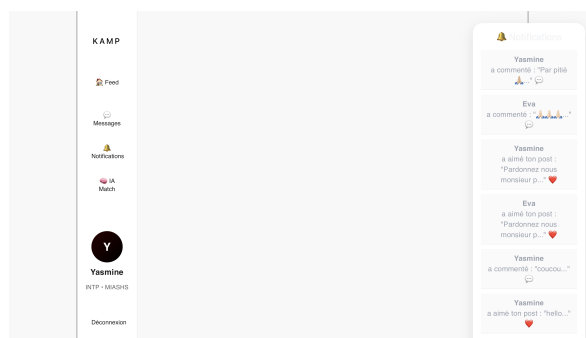


FIGURE 4 – Page des notifications

8. L'utilisateur peut consulter la page IA Match pour voir des profils compatibles.
9. Il peut aussi envoyer des messages dans les salons de discussion.

Annexe C : Manuel utilisateur

Pour utiliser l'application, l'utilisateur doit d'abord se créer un compte ou se connecter à un compte existant.

Après la connexion, il doit compléter son profil en indiquant son prénom, son type MBTI, sa formation et éventuellement une photo.

Une fois le profil créé, l'utilisateur arrive sur le fil d'actualité. Il peut publier un message, consulter les publications des autres utilisateurs, liker et commenter.

La barre latérale permet de naviguer dans l'application :

- le bouton Feed permet de revenir au fil d'actualité ;
- le bouton Messages permet d'accéder aux salons de discussion ;
- le bouton Notifications permet d'afficher les notifications ;
- le bouton IA Match permet de voir les profils compatibles ;
- le bouton Déconnexion permet de quitter le compte.